



ADMISSION DOCUMENT AND APPLICANT MANAGEMENT SYSTEM (A D A M S)

Design and Development of an Automated Admission Management and Tracking System

A Final Project Documentation

Presented to the Faculty of the
College of Computer and Information Sciences
Polytechnic University of the Philippines

In Partial Fulfilment of the Requirements for the Degree
Bachelor of Science in Information Technology

by

San Pedro, Jonathan C.

Balakwit, Joan S.

Cioco, Gracheil M.

Gravoso, Catherine S.

Pamintuan, Kriszia Mae H.

BSIT-OUMN 2-3
Information Management
Target Client: PUP Open University System Registrar's Office
June 2026



Republic of the Philippines

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Open University System



ACKNOWLEDGEMENTS

We wish to express our sincere gratitude to the faculty of the Polytechnic University of the Philippines Open University System for the guidance, feedback, and project requirements that shaped this final Information Management documentation. We also acknowledge the PUP Open University System Registrar's Office as the target client and process reference for the proposed admission document management prototype.

We further thank our classmates, resource persons, and team members who contributed to system analysis, database design, user interface preparation, documentation drafting, and testing. Their technical and administrative inputs helped complete the Admission Document and Applicant Management System (ADAMS) as a database-driven final project.

J.C.S. / J.S.B. / G.M.C. / C.S.G. / K.M.H.P.



TABLE OF CONTENTS

Title Page	
Acknowledgements	
Table of Contents	
List of Tables	
List of Figures	
Chapter 1. Project Context	
1.1 Introduction	
1.2 Project Overview	
1.3 Target Client and Users	
1.4 Problem Identification	
1.5 Business Rules	
1.6 Objectives of the Study/System	
1.7 Scope and Limitations	
Chapter 2. Database Design and Documentation Requirements	
2.1 Current Form and Proposed Form	
2.2 Entities and Attributes	
2.3 Normalization	
2.4 Entity Relationship Diagram (ERD)	
2.5 Enhanced Entity Relationship Diagram (EERD)	
2.6 Data Dictionary	
Chapter 3. Full Application and Functional Requirements	
3.1 System Architecture	
3.2 N Screenshots of User Interface	
3.3 Functional Requirements and User Manual	
3.4 Non-Functional Requirements	
Chapter 4. SQL Implementation	
4.1 SQL Query Requirements	
4.2 Simple SQL Queries	
4.3 Moderate SQL Queries	
4.4 Difficult SQL Queries	
4.5 SQL Code / DDL Schema	
Chapter 5. Program Requirements and Deployment	
5.1 Hardware Requirements	
5.2 Software Requirements	
5.3 Deployment and Backup Requirements	
References	
Appendices	
Appendix A. Source Code and Database Package Contents	



LIST OF TABLES

Table 1. Problem identification and database response
Table 2. ADAMS entities and attributes
Table 3. Unnormalized form example
Table 4. First normal form example
Table 5. Second normal form decomposition
Table 6. Third normal form decomposition
Table 7. ERD relationship cardinalities
Table 8. EERD element explanation
Table 9. Data dictionary
Table 10. Application modules and functions
Table 11. Functional requirements and user manual
Table 12. Non-functional requirements
Table 13. Hardware requirements
Table 14. Software requirements
Table 15. Source code and database package contents



LIST OF FIGURES

- Figure 1. Current CAEPUP-OUS application form
- Figure 2. Proposed ADAMS admission form
- Figure 3. ADAMS ERD summary view
- Figure 4. ADAMS ERD detailed implementation view
- Figure 5. ADAMS enhanced entity relationship diagram
- Figure 6. ADAMS login page
- Figure 7. ADAMS dashboard page
- Figure 8. Student management page
- Figure 9. Student archives page
- Figure 10. Category management page
- Figure 11. Requirement management page
- Figure 12. Activity logs page
- Figure 13. User management page



Chapter 1. Project Context

1.1 Introduction

Admission offices receive and validate many student documents during application and enrolment. When information is monitored through manual forms, separate folders, or scattered files, staff may spend more time searching for records than verifying the actual requirements. The ADAMS project responds to this concern by proposing a database-driven admission document and applicant management system for the PUP Open University System.

The system focuses on the Information Management requirement of designing a normalized relational database and connecting it to a working prototype application. The documentation therefore presents both the database design and the full application features, including entities, normalization, ERD, EERD, data dictionary, user interface screenshots, SQL queries, program requirements, functional requirements, and SQL DDL schema.

1.2 Project Overview

The Admission Document and Applicant Management System (ADAMS) is a database-driven prototype for managing student admission records and document requirements for the PUP Open University System. The system organizes applicants/students, student categories, required document types, uploaded requirement files, user accounts, activity logs, printing records, password reset records, and archived student records into a structured relational database.

The main purpose of ADAMS is to replace scattered manual monitoring with a database that can answer operational questions: who the student is, what category the student belongs to, what requirements are expected, which requirements have already been uploaded, which users performed changes, which records were printed, and which student records were archived. The database component is the core of the system because it uses primary keys, foreign keys, unique constraints, bridge tables, and logical relationships to support accurate retrieval, completeness checking, accountability, and reporting.

1.3 Target Client and Users

The target client of ADAMS is the PUP Open University System Registrar's Office. The system is intended to support registrar-related admission document monitoring, applicant profile management, document checklist tracking, report printing, and record archiving.

The submitted user scope includes only Superadmin and Staff roles. Applicants/students are maintained as admission records and are not included as login users in this final documentation scope.

- **Superadmin:** manages system-wide user accounts, settings, categories, requirements, logs, and reports.
- **Staff:** handles routine record searching, updating, uploading, monitoring, printing, and archiving subject to granted access.

1.4 Problem Identification

The major problem addressed by ADAMS is the difficulty of managing admission document records when student information, required documents, uploaded files, and monitoring logs are handled manually or inconsistently. Without a relational database, the office may experience duplicate records, incomplete document tracking, unclear accountability, and difficulty identifying which requirements remain pending.

Table 1. Problem identification and database response



Identified Problem	Operational Effect	Database Response
Manual or scattered recording	Student records and requirement statuses become difficult to search and verify.	Create searchable students, student_categories, requirement_types, and student_requirements tables.
Different requirements per student category	The office may apply the wrong checklist to the wrong student group.	Use category_requirements to define the required documents per category.
Incomplete or pending uploaded requirements	Staff may not quickly know which requirements are missing, pending, rejected, or insufficient.	Use student_requirements with status, file_name, file_path, upload_date, and uploaded_at.
Unclear accountability	It may be difficult to identify who performed an update, upload, print, or archive action.	Use users, activity_log, uploaded_by, archived_by, and print_logs.
Need to preserve removed records	Deleted or inactive student records may be lost without historical reference.	Use archived_students to keep student snapshots and archive details.

1.5 Business Rules

- Each student/applicant belongs to one student category when classified.
- A student category may require many admission requirement types.
- A requirement type may be required by many student categories.
- A student may have many requirement records.
- Each student requirement refers to one requirement type.
- A student may have document records attached to the student profile.
- Authorized users perform system actions and uploads.
- Every major system action must be traceable through activity logs or related audit fields.
- Student report printing must be recorded for accountability.
- Archived students are kept as historical snapshots instead of being permanently lost.
- The mapping between student categories and requirement types must be maintained through a bridge table to avoid repeating checklist data.
- Only valid requirement statuses may be recorded, such as PENDING, SUBMITTED, VERIFIED, APPROVED, INSUFFICIENT, or REJECTED.

1.6 Objectives of the Study/System

General Objective. The general objective of ADAMS is to develop a database-driven admission document management prototype that helps the office record, monitor, verify, retrieve, print, and archive student admission records and requirements more accurately and efficiently.

Specific Objectives. The system specifically aims to:

- Create a structured database for users, students, student categories, requirement types, category requirements, student requirements, document records, activity logs, print logs, password reset tokens, and archived student records.
- Define which requirement types are applicable to each student category through a bridge table.
- Record student-specific requirement statuses, uploaded files, upload dates, and uploading users.
- Allow authorized users to manage student records, requirements, uploads, printing, password recovery, and archiving activities.
- Apply normalization principles to reduce redundancy, remove repeating groups, and improve consistency.



-
- Present a defensible database design through ERD, EERD, data dictionary, SQL queries, and SQL schema code.
 - Demonstrate useful SQL queries for requirement lists, student checklist monitoring, missing requirement detection, activity tracking, and reporting.

1.7 Scope and Limitations

The scope of ADAMS covers admission-related document tracking, student profile maintenance, category and requirement setup, file upload monitoring, user account administration, activity logging, report printing, password reset token tracking, and archiving. It is designed as a final project prototype, not as a fully deployed university-wide production system.

The current prototype depends on the correctness of encoded applicant information and uploaded files. Integration with official university authentication, payment systems, student information systems, and external verification agencies is outside the present scope, but the relational design can be extended to support these integrations in future versions.



Chapter 2. Database Design and Documentation Requirements

2.1 Current Form and Proposed Form

The current admission form captures applicant profile data, contact information, emergency contact, academic information, employment information, and chosen academic program. ADAMS converts these information areas into structured database fields and upload-monitoring workflows so that the office can search, validate, and report admission records more efficiently.

Current Form

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES ADMISSION AND REGISTRATION SERVICES CAEPUP-OUS 2024 STA. MESA, MANILA	
CAEPUP 2023 APPLICATION FORM	
REFERENCE NUMBER: 0000000000	DATE OF APPLICATION: 03/04/2024
PERSONAL INFORMATION:	
Complete Name: RODRIGO, JOSEPH RICHARD ROSE	
Sex: M	Date of Birth: 03/03/2004
Civil Status: SINGLE	
CONTACT INFORMATION:	
Email Address: jrodrigo03@gmail.com	Contact Number/s: 0905-12345678
Address: Block 11 Lot 5 Waling Waling St. San Roqueville Proper Village, TAA, City of Las Piñas, Philippine	
IN CASE OF EMERGENCY, Please Notify:	
Contact Person: Richard Rose	Contact Number/s: 0905-12345678
Address: Block 11 Lot 5 Waling Waling St. San Roqueville Proper Village, TAA, City of Las Piñas, Philippine	
ACADEMIC INFORMATION (LAST SCHOOL ATTENDED):	
Mode of Learning: College/Bachelor's Degree	
School Name: Central State University - Bataan Campus	
School Address: San Jose St. Marikina City, Cavite	Year Graduated: 2021
EMPLOYMENT INFORMATION (OPTIONAL):	
Company: All House Flashed Global South (Self Employed)	Contact Number/s: 0905-12345678
Address: Vista Mall 28 Extension, Road Division Ave, Las Piñas	
CHOSEN ACADEMIC PROGRAM:	
BSBAHRMOUMN-BACHELOR OF SCIENCE IN BUSINESS ADMINISTRATION MAJOR IN HUMAN RESOURCE MANAGEMENT (OU MANILA)	
CONFORME:	

Figure 1. Current CAEPUP-OUS application form

The proposed ADAMS form supports structured entry of personal information, contact details, emergency contact, academic information, employment information, and academic program preferences. The design is aligned with the proposed database by separating student identity, category classification, requirements, files, and status tracking.



Proposed Form



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
OPEN UNIVERSITY SYSTEM
PUPOUSAAET 2026 STA. MESA, MANILA

Applicant Photo:

2x2
Choose file

Your valid, scanned 2x2 inch photo in JPEG format. Maximum file size allowed is 300 kilobytes (KB).

ADMISSION FORM

PERSONAL INFORMATION:

Complete Name: *

Your complete name based on your PSA birth certificate

Sex: *

☐ Male ☐ Female

Your biological and physiological characteristic

Date of Birth: *

Height: cm

Weight: kg

Civil Status: *

Single

Place of Birth: *

Religion: *

Disability: Not Applicable

CONTACT INFORMATION:

Address: *

Type here the address including Unit, Number, Street and Subdivision/Barangay (Example: Unit 101 #88 Teresa St., Sta. Mesa)

Email Address: *

Contact Number/s: *

This is your registered e-mail address:

IN CASE OF EMERGENCY, Please Notify:

Contact Person: *

Contact Number/s: *

☐ Same as my Address

Address: *

Type here the address including Unit, Number, Street and Subdivision/Barangay (Example: Unit 101 #88 Teresa St., Sta. Mesa)

ACADEMIC INFORMATION (LAST SCHOOL ATTENDED):

Mode of Learning: *

Year Graduated: *

2025

School Name: *

School Address: *

EMPLOYMENT INFORMATION (OPTIONAL):

Company:

Contact Number/s:

Company Address:

ACADEMIC PROGRAM PREFERENCE:

You are applying *

PUPOUSAAET

Branch/Campus: *

Sta. Mesa, Manila

Figure 2. Proposed ADAMS admission form

2.2 Entities and Attributes

Table 2. ADAMS entities and attributes

Entity/Table	Primary Key	Major Attributes	Purpose
users	user_id	username, password, email, full_name, phone, role, is_active, created_at, last_login	Stores authorized accounts and access roles.



student_categories	category_id	category_name, sort_order	Stores reusable student classifications such as Freshman, Transferee, Returning, or similar categories.
students	student_id	user_id, student_name, last_name, first_name, middle_name, birth_date, email, category_id, remarks, phone	Stores applicant/student profile information.
requirement_types	requirement_id	requirement_name, description	Stores the master list of admission document requirements.
category_requirements	category_id + requirement_id	category_id, requirement_id	Bridge table that defines which requirements apply to each student category.
student_requirements	student_requirement_id	student_id, requirement_id, file_name, file_path, file_content, mime_type, file_size, uploaded_by, status, upload_date, uploaded_at	Tracks student-specific requirement status and uploaded files.
documents	document_id	student_id, document_type, file_path, tracking_number, status, date_submitted, remarks, student_pk	Stores additional document records attached to student profiles.
activity_log	log_id	user_id, username, action, module, description, log_time	Records major user actions for accountability.
print_logs	log_id	ref_no, student_id, document_type, printed_by, printed_by_name, printed_at, campus_code	Records printed student reports or system reports.
password_reset_tokens	id	user_id, token, expires_at, used, created_at	Stores password recovery tokens and token status.
archived_students	archive_id	student_id, student_name, last_name, first_name, middle_name, email, phone, birth_date, category_id, remarks, archived_at, archived_by	Stores snapshots of archived student records.

2.3 Normalization

Normalization was applied to convert a logbook-style admission record into a cleaner relational database. In a manual setup, one record may contain student details, category details, multiple requirement names, multiple status values, and upload information in the same row. This creates redundancy and update errors. The normalized design separates repeated data into related tables.

Unnormalized Form (UNF)

In UNF, multiple requirements and upload details may appear in one wide record. This violates the rule that fields should contain atomic values.

Table 3. Unnormalized form example

Student Name	Category	Requirement 1	Status 1	File 1	Requirement 2	Status 2	File 2	Remarks
--------------	----------	---------------	----------	--------	---------------	----------	--------	---------



Reyes, Ana	Freshman	Birth Certificate	SUBMITTED	birth.pdf	Valid ID	PENDING	N/A	For verification
Santos, Mark	Transferee	Transcript	PENDING	N/A	Good Moral	SUBMITTED	goodmoral.pdf	Incomplete

First Normal Form (1NF)

In 1NF, repeating groups are removed. Each requirement is represented as a separate row with a single requirement value and a single status value.

Table 4. First normal form example

Student Name	Category	Requirement	Status	File Name	Uploaded By	Upload Date
Reyes, Ana	Freshman	Birth Certificate	SUBMITTED	birth.pdf	admin	2026-06-20
Reyes, Ana	Freshman	Valid ID	PENDING	N/A	admin	
Santos, Mark	Transferee	Transcript	PENDING	N/A	registrar	
Santos, Mark	Transferee	Good Moral	SUBMITTED	goodmoral.pdf	registrar	2026-06-20

Second Normal Form (2NF)

In 2NF, data is separated based on its main purpose. Student details, category details, requirement type details, and student requirement records are separated to reduce partial dependency and repeated fields.

Table 5. Second normal form decomposition

Table	Primary Key	Purpose
students	student_id	Stores student/applicant identity once.
student_categories	category_id	Stores reusable category names such as Freshman, Transferee, Returning, and similar groups.
requirement_types	requirement_id	Stores reusable requirement names such as Birth Certificate or Valid ID.
category_requirements	category_id + requirement_id	Defines which requirements apply to each category.
student_requirements	student_requirement_id	Stores requirement status and upload details per student and requirement.

Third Normal Form (3NF)

In 3NF, remaining transitive or repeated operational data is separated into supporting tables. Users, activity logs, print logs, password reset tokens, documents, and archived student records support accountability, reporting, and security without repeating the same details inside the student table.

Table 6. Third normal form decomposition

Final Table	Key Fields	Reason for Separation
users	user_id, username	Stores authorized accounts and roles once.



activity_log	log_id, user_id	Stores system actions separately for traceability.
print_logs	log_id, ref_no	Stores printing history separately from student records.
password_reset_tokens	id, user_id, token	Stores password recovery tokens separately for security.
documents	document_id, student_pk	Stores document records linked to a student.
archived_students	archive_id, student_id	Stores historical snapshots of archived student records.

The students_backup table found in the database package is excluded from the normalized operational design because it is a backup/support table, not part of the main application model.

2.4 Entity Relationship Diagram (ERD)

The ERD presents the implementation view of the ADAMS database. It includes actual tables, major attributes, primary keys, foreign keys, unique constraints, relationship labels, and cardinality. The detailed view below corresponds to the relational tables documented in the data dictionary and DDL schema.

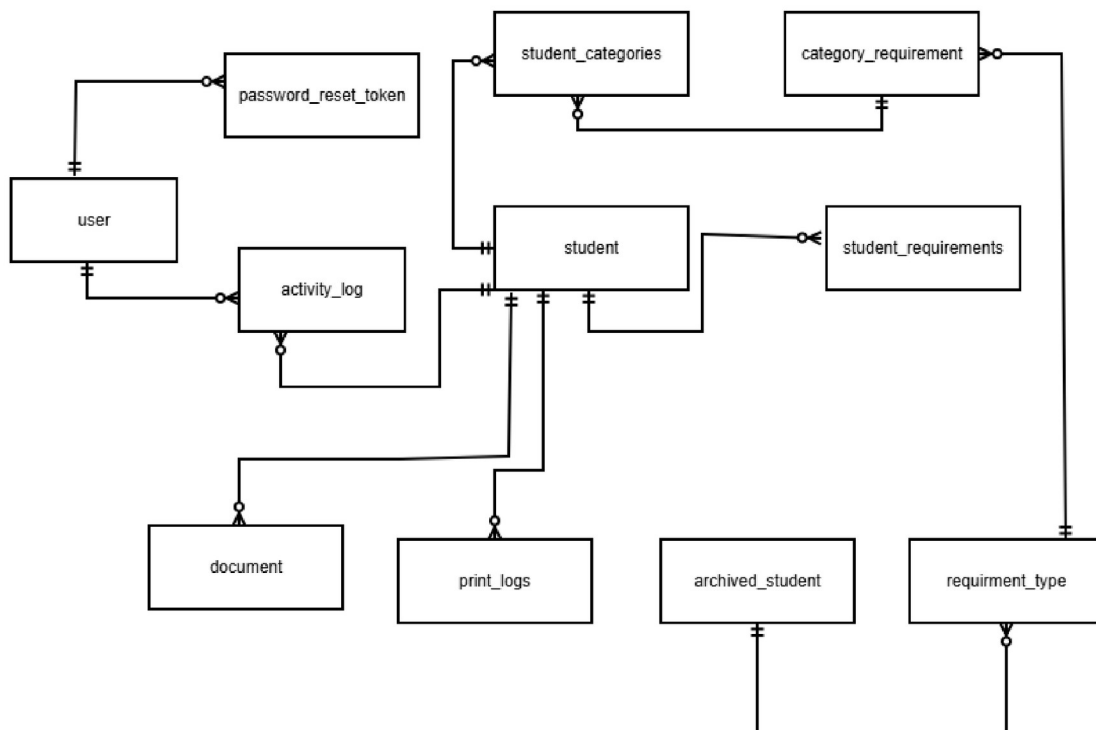


Figure 3. ADAMS ERD summary view

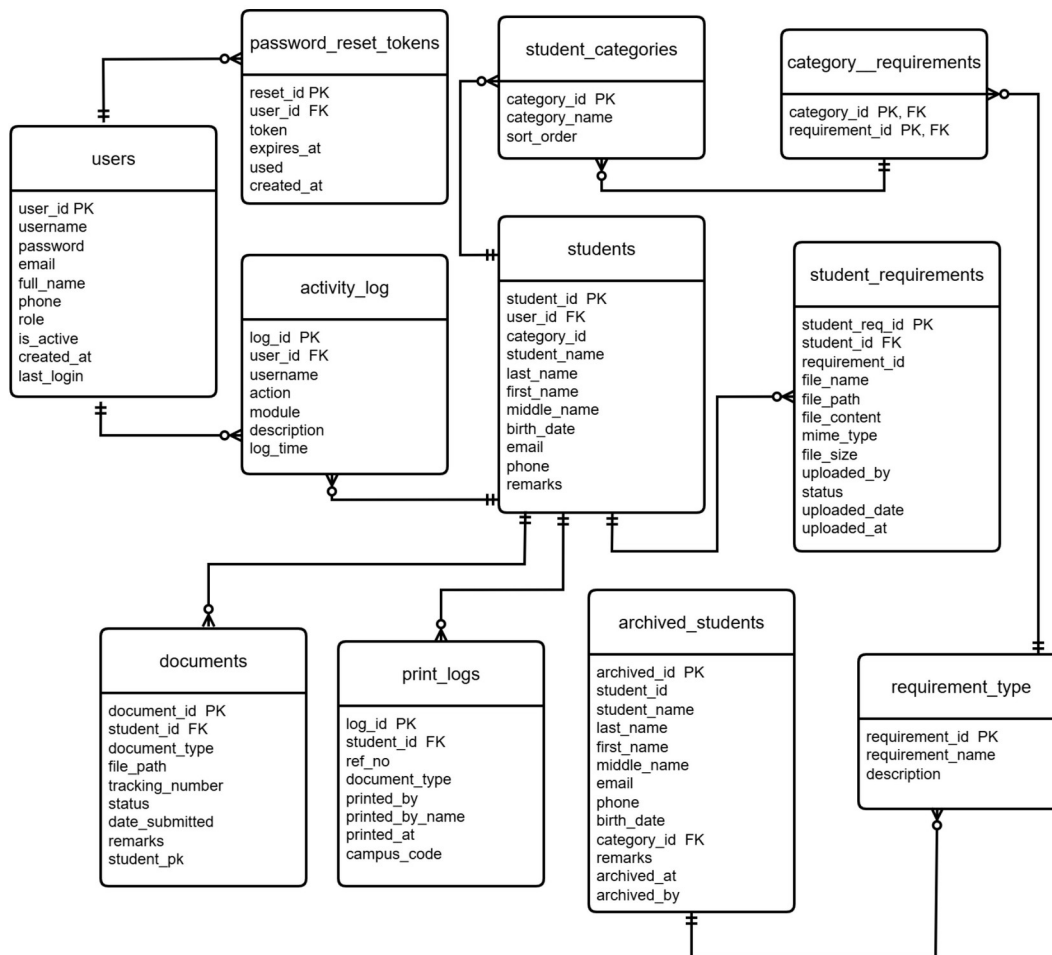


Figure 4. ADAMS ERD detailed implementation view

Table 7. ERD relationship cardinalities

Relationship	Cardinality	Explanation
student_categories -> students	1:M	One category can classify many students; each student belongs to one category when assigned.
student_categories -> category_requirements	1:M	One category may require many requirement types.
requirement_types -> category_requirements	1:M	One requirement type may appear in many category checklists.
students -> student_requirements	1:M	One student may have many requirement records.



requirement_types -> student_requirements	1:M	One requirement type may be assigned to many student requirement records.
students -> documents	1:M	One student may have many document records.
users -> activity_log	1:M	One user may create many activity log entries.
users -> print_logs	1:M	One user may print many records/reports.
users -> password_reset_tokens	1:M	One user may request many password reset tokens.

2.5 Enhanced Entity Relationship Diagram (EERD)

The EERD presents the conceptual enhanced model using Chen-style notation. It emphasizes specialization/generalization for the two authorized system roles, namely Superadmin and Staff, and also shows student classification through student categories. To keep the diagram readable, it shows major attributes only; the complete SQL fields are shown in the ERD, data dictionary, and DDL schema.

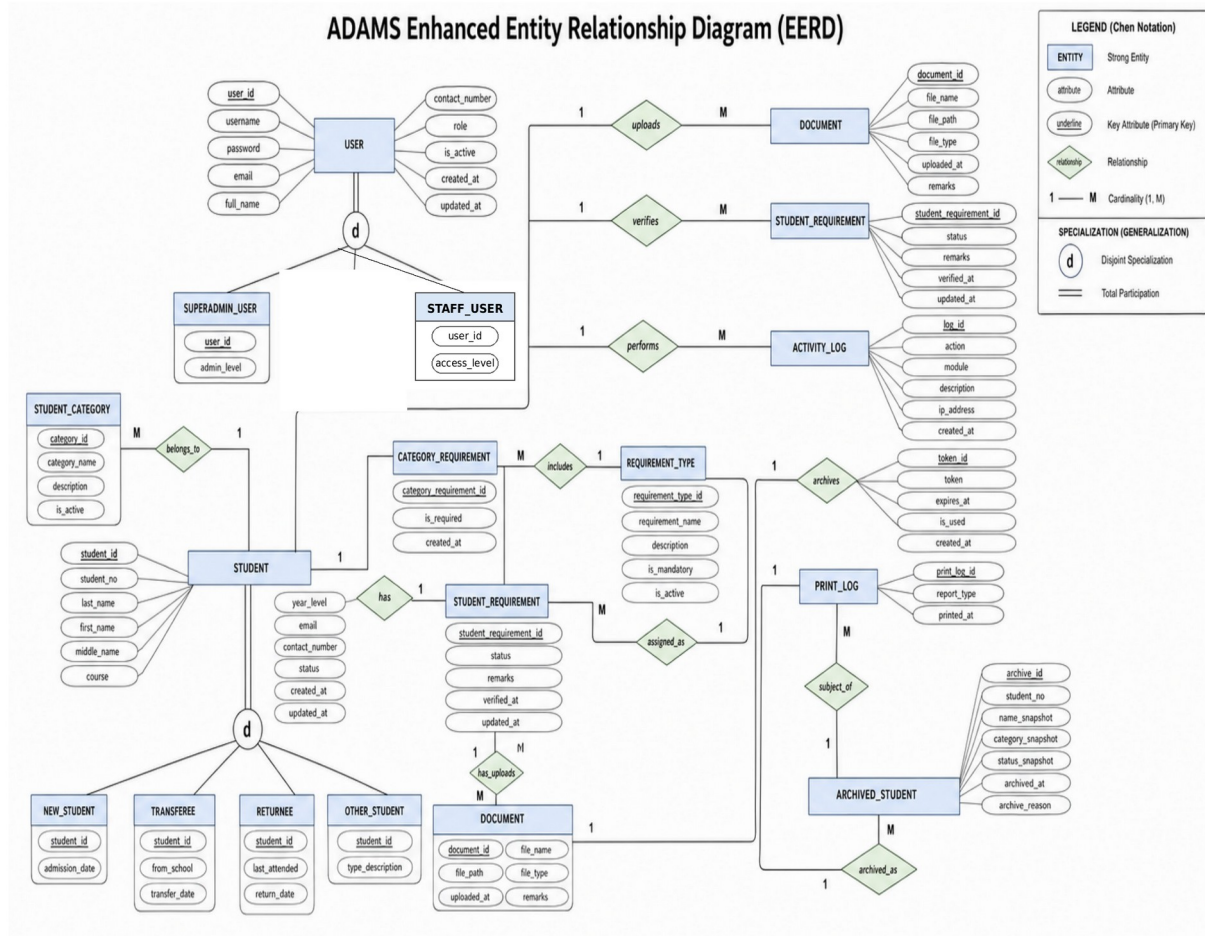


Figure 5. ADAMS enhanced entity relationship diagram



Table 8. EERD element explanation

EERD Element	Meaning in ADAMS
USER specialization	Users are classified only as SUPERADMIN_USER or STAFF_USER based on the role field.
STUDENT specialization	Students may be conceptually classified into categories such as Freshman, Sophomore, Junior, Senior, Transferee, and Returning. In the database this is implemented through student_categories and students.category_id.
Relationship diamonds	Diamonds show actions or associations such as classifies, has, assigned_as, uploads, records, requests, prints, archives, and archived_as.
Cardinality labels	1 and M labels show whether one record can relate to many records in another entity.
Primary key ovals	Underlined ovals indicate key attributes in Chen notation.

2.6 Data Dictionary

The data dictionary below is based on the ADAMS operational tables. The backup table students_backup is excluded from the main documentation because it is not part of the core application model.

Table 9. Data dictionary

Table	Field	Data Type	Key/Constraint	Description
users	user_id	INT	PK	Unique user identifier.
users	username	VARCHAR(50)	UK, NOT NULL	Login username.
users	password	VARCHAR(100)	NOT NULL	Stored password/hash value.
users	email	VARCHAR(100)	NULL	User email address.
users	full_name	VARCHAR(100)	NULL	Full name of user.
users	phone	VARCHAR(20)	NULL	Contact number.
users	is_active	TINYINT(1)	DEFAULT 1	Indicates active account.
users	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation date.
users	last_login	DATETIME	NULL	Latest login time.
users	role	VARCHAR(20)	CHECK	User role: superadmin or staff.
student_categories	category_id	INT	PK	Unique category identifier.
student_categories	category_name	VARCHAR(100)	UK, NOT NULL	Name of student category.
student_categories	sort_order	INT	DEFAULT 0	Ordering value for display.
students	student_id	INT	PK	Unique student identifier.



students	user_id	INT	FK, NULL	Optional linked user account.
students	student_name	VARCHAR(100)	UK with birth_date	Full student name.
students	last_name	VARCHAR(100)	NOT NULL	Student last name.
students	first_name	VARCHAR(100)	NOT NULL	Student first name.
students	middle_name	VARCHAR(100)	NULL	Student middle name.
students	birth_date	DATE	NULL	Date of birth.
students	email	VARCHAR(100)	NULL	Email address.
students	category_id	INT	FK	Student category.
students	remarks	VARCHAR(255)	NULL	Remarks or notes.
students	phone	VARCHAR(20)	NULL	Phone number.
requirement_types	requirement_id	INT	PK	Unique requirement type identifier.
requirement_types	requirement_name	VARCHAR(100)	UK, NOT NULL	Name of requirement.
requirement_types	description	VARCHAR(255)	NULL	Requirement description.
category_requirements	category_id	INT	PK/FK	References student_categories.
category_requirements	requirement_id	INT	PK/FK	References requirement_types.
student_requirements	student_requirement_id	INT	PK	Unique student requirement record.
student_requirements	student_id	VARCHAR(20)	NOT NULL, UK with requirement_id	Logical student reference used by the application.
student_requirements	requirement_id	INT	NOT NULL, Indexed	Requirement type reference.
student_requirements	file_name	VARCHAR(255)	NULL	Uploaded file name.
student_requirements	file_path	VARCHAR(500)	NULL	Uploaded file path.
student_requirements	file_content	LONGBLOB	NULL	Binary file content.
student_requirements	mime_type	VARCHAR(100)	NULL	File MIME type.
student_requirements	file_size	BIGINT	NULL	File size.
student_requirements	uploaded_by	INT	NULL	User who uploaded the file, if recorded.
student_requirements	status	VARCHAR(50)	Indexed	Requirement status.
student_requirements	upload_date	DATE	NULL	Upload date.
student_requirements	uploaded_at	DATETIME	NULL	Upload timestamp.
documents	document_id	INT	PK	Unique document record.
documents	student_id	VARCHAR(20)	NULL	Student number or external student



				reference.
documents	document_type	VARCHAR(50)	NULL	Document type label.
documents	file_path	VARCHAR(255)	NULL	Document file path.
documents	tracking_number	VARCHAR(50)	NULL	Tracking/reference number if used.
documents	status	VARCHAR(20)	NULL	Document status.
documents	date_submitted	DATETIME	NULL	Submission date.
documents	remarks	VARCHAR(255)	NULL	Remarks.
documents	student_pk	INT	NOT NULL, FK	References students.student_id.
activity_log	log_id	INT	PK	Unique activity log identifier.
activity_log	user_id	INT	FK	User who performed the action.
activity_log	username	VARCHAR(100)	NOT NULL	Username at time of action.
activity_log	action	VARCHAR(50)	CHECK	Action type such as ADD, UPDATE, DELETE, REPLACE, UPLOAD, ARCHIVE.
activity_log	module	VARCHAR(50)	NOT NULL	Module affected.
activity_log	description	TEXT	NOT NULL	Action description.
activity_log	log_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date/time of activity.
print_logs	log_id	INT	PK	Unique print log.
print_logs	ref_no	VARCHAR(100)	UK	Unique reference number.
print_logs	student_id	INT	FK, NULL	Student printed, if applicable.
print_logs	document_type	VARCHAR(50)	CHECK	Report/document type.
print_logs	printed_by	INT	FK, NULL	User who printed.
print_logs	printed_by_name	VARCHAR(100)	NULL	Name of printing user.
print_logs	printed_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Print date/time.
print_logs	campus_code	VARCHAR(10)	DEFAULT MN0	Campus code.
password_reset_tokens	id	INT	PK	Unique token record.
password_reset_tokens	user_id	INT	FK	User requesting reset.
password_reset_tokens	token	VARCHAR(64)	UK, NOT NULL	Reset token.
password_reset_tokens	expires_at	DATETIME	NOT NULL	Expiration timestamp.
password_reset_tokens	used	TINYINT(1)	DEFAULT 0	Usage flag.
password_reset_tokens	created_at	DATETIME	DEFAULT	Creation timestamp.



			CURRENT_TIMESTAMP	
archived_students	archive_id	INT	PK	Unique archive record.
archived_students	student_id	INT	Logical reference	Original student identifier.
archived_students	student_name	VARCHAR(255)	NULL	Archived full name.
archived_students	last_name	VARCHAR(100)	NULL	Archived last name.
archived_students	first_name	VARCHAR(100)	NULL	Archived first name.
archived_students	middle_name	VARCHAR(100)	NULL	Archived middle name.
archived_students	email	VARCHAR(255)	NULL	Archived email.
archived_students	phone	VARCHAR(50)	NULL	Archived phone.
archived_students	birth_date	DATE	NULL	Archived birth date.
archived_students	category_id	INT	Logical/FK	Archived category.
archived_students	remarks	TEXT	NULL	Archived remarks.
archived_students	archived_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Archive timestamp.
archived_students	archived_by	INT	Logical/FK	User who archived record.



Chapter 3. Full Application and Functional Requirements

3.1 System Architecture

ADAMS follows a web-based Java application structure using JSP/Servlets, DAO classes, model classes, utility classes, and a MySQL or MariaDB relational database. The application layer handles forms, validation, user actions, file upload, reporting, and printing. The database layer stores normalized data and supports SQL queries for monitoring and reporting.

Table 10. Application modules and functions

Module	Main Function
Login and Authentication	Allows authorized users to log in, log out, and recover passwords through reset tokens.
Dashboard	Displays summary counts, monitoring indicators, and reports for management review.
Student Management	Allows adding, searching, editing, viewing, printing, and archiving student/applicant records.
Category Management	Allows maintaining student categories and their display order.
Requirement Management	Allows maintaining requirement types and mapping requirements to categories.
Upload / Student Requirements	Allows uploading files and updating requirement status per student.
Activity Logs	Allows the Superadmin to review traceable system actions and accountability records.
Print Logs and Reports	Records generated reports and reference numbers.
User Management	Allows the Superadmin to maintain system user accounts, roles, and activity status.
Archives	Keeps historical snapshots of archived student records.

3.2 N Screenshots of User Interface

The following screenshots show the major pages of the ADAMS full application prototype. These screens demonstrate that the database design is connected to functional pages for login, dashboard monitoring, student management, archives, category setup, requirement setup, activity logs, and user management.



Prototype - Screen Shots

The screenshot shows the login page for the ADAMS system. At the top, there is a header with the Polytechnic University of the Philippines logo on the left, the university name in the center, and a circular seal on the right. Below the header, the text "ADAMS" is prominently displayed in a large, bold, dark red font. Underneath "ADAMS", the full name "Automated Document Admission Management System (ADAMS)" is written in a smaller, grey font. The login form consists of two input fields: "Username" with a person icon and "Password" with a lock icon. Both fields are empty and have a light grey border.

Figure 6. ADAMS login page

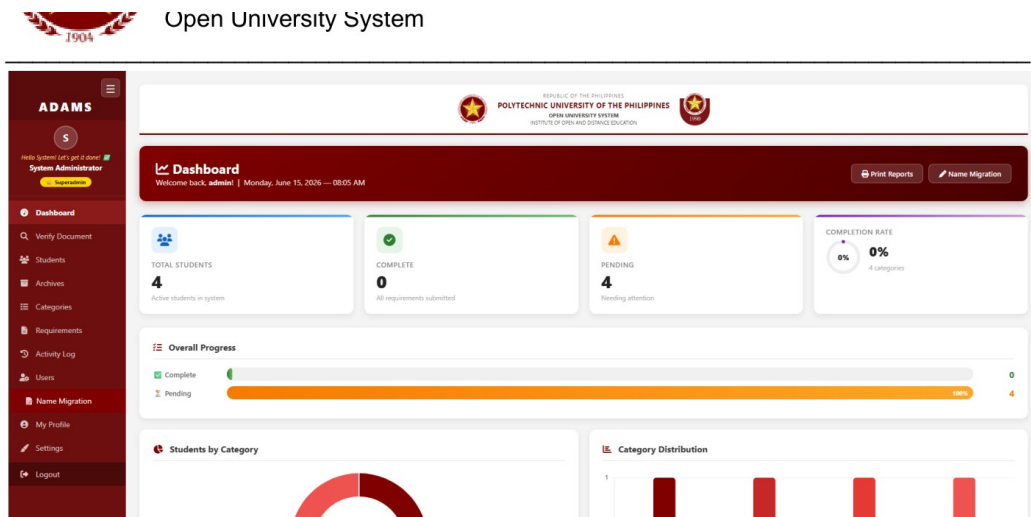


Figure 7. ADAMS dashboard page



Dashboard

ADAMS

My System Group to verify
System Administrator

Dashboard

Verify Document

Students

Archives

Categories

Requirements

Activity Log

Users

Name Migration

My Profile

Settings

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
OPEN UNIVERSITY SYSTEM
INSTITUTE OF OPEN AND DISTANCE EDUCATION

Logged in as: admin
Role: Superadmin

Show 10 entries

+ Add Student

Search name or email

All Categories

Search

☐	#	ID	Name	Email	Category	Phone	Status	Progress	Actions
☐	1	3	GARCIA, Liza	liza.garcia@example.com	Graduate Student	09175531234	Missing	0%	View Edit Docs Print Archive
☐	2	4	PAMINTUAN, Krizia	kriziamhampamintuan@iskolamgabay.pup.edu.ph	Freshman	09178880000	Missing	0%	View Edit Docs Print Archive
☐	3	5	PAMINTUAN, Krizia	kriziamhampamintuan@iskolamgabay.pup.edu.ph	Returning Student	09178880000	Missing	0%	View Edit Docs Print Archive
☐	4	6	PAMINTUAN, Krizia	kriziamhampamintuan@iskolamgabay.pup.edu.ph	Transferee	09178880000	Missing	0%	View Edit Docs Print Archive

Page 1 of 1 | Showing 1-4 of 4 students

Figure 8. Student management page



ADAMS

Welcome back, System!
System Administrator

Dashboard

Verify Document

Students

Archives

Categories

Requirements

Activity Log

Users

Name Migration

My Profile

Settings

Logout

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
OPEN UNIVERSITY SYSTEM
INSTITUTE OF OPEN AND DISTANCE EDUCATION

Logged in as: admin
Role: superadmin

Student Archives

2 Total Archived

2 Showing

Search name or email

All Categories

Search

☐	#	Student ID	Name	Email	Category	Archived Date	Archived By	Actions
☐	1	2	SANTOS, Mark R.	mark.santos@example.com	Transferee	2020-06-15 05:50	admin	View Edit Docs Print Archive
☐	2	1	DELA CRUZ, Ana S.	ana.delacruz@example.com	Freshman	2020-06-14 17:01	admin	View Edit Docs Print Archive

Figure 9. Student archives page



ADAMS

5

Hi! System! Let's get it done!

System Administrator

Logout

Dashboard

Verify Document

Students

Archives

Categories

Requirements

Activity Log

Users

Name Migration

My Profile

Settings

Logout

REPUBLIC OF THE PHILIPPINES

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

OPEN UNIVERSITY SYSTEM

INSTITUTE OF OPEN AND DISTANCE EDUCATION

Logged in as: admin
Role: Superadmin

Category Name

#	Category Name	Actions
1	Freshman	Edit Delete
2	Transferee	Edit Delete
3	Graduate Student	Edit Delete
4	Returning Student	Edit Delete

Figure 10. Category management page



ADAMS

5

Good afternoon, System!

System Administrator

Logout

Dashboard

Verify Document

Students

Archives

Categories

Requirements

Activity Log

Users

Name Migration

My Profile

Settings

Logout

REPUBLIC OF THE PHILIPPINES

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

OPEN UNIVERSITY SYSTEM

INSTITUTE OF OPEN AND DISTANCE EDUCATION

Logged in as: admin
Role: Superadmin

#	Requirement Name	Description	Actions
1	Birth Certificate	Official PSA or NSO birth certificate.	Edit Delete
2	Form 138 / Report Card	Latest report card or senior high school academic record.	Edit Delete
3	Good Moral Certificate	Certificate of good moral character issued by the previous school.	Edit Delete
4	2x2 ID Picture	Recent 2x2 identification photo.	Edit Delete
5	Valid ID	Government-issued, school-issued, or other valid identification document.	Edit Delete
6	Transcript of Records	Official transcript for transferee or graduate applicant.	Edit Delete

Figure 11. Requirement management page



Requirements

#	Date & Time	User	Action	Module	Description
1	Jun 15, 2026 09:50 AM	admin	ARCHIVE	Students	Archived student: "SANTOS, Mark R."
2	Jun 14, 2026 09:01 PM	admin	ARCHIVE	Students	Archived student: "DELA CRUZ, Ana S."
3	Jun 14, 2026 09:01 PM	admin	RESTORE	Students	Restored student: "DELA CRUZ, Ana S."
4	Jun 14, 2026 04:58 PM	admin	UPDATE	Students	Updated student: "RAMINTUAN, Kriszia" (ID: 5) Email: krisziamaehpamintuan@iskolamgbayan.pup.edu.ph
5	Jun 14, 2026 04:58 PM	admin	UPDATE	Students	Updated student: "RAMINTUAN, Kriszia" (ID: 5) Email: krisziamaehpamintuan@iskolamgbayan.pup.edu.ph
6	Jun 07, 2026 02:59 AM	admin	ADD	Students	Added new student: "RAMINTUAN, Kriszia" Email: krisziamaehpamintuan@iskolamgbayan.pup.edu.ph
7	Jun 07, 2026 02:18 AM	admin	ADD	Students	Added new student: "RAMINTUAN, Kriszia" Email: krisziamaehpamintuan@iskolamgbayan.pup.edu.ph
8	Jun 05, 2026 06:46 AM	admin	ARCHIVE	Students	Archived student: "DELA CRUZ, Ana S."
9	Jun 05, 2026 06:45 AM	admin	UPDATE	Students	Batch category update — 1 student(s) updated to category ID: 1
10	Jun 05, 2026 06:43 AM	admin	UPDATE	Students	Batch category update — 1 student(s) updated to category ID: 1
11	Jun 05, 2026 05:42 AM	admin	ADD	Students	Added new student: "RAMINTUAN, Kriszia" Email: krisziamaehpamintuan@iskolamgbayan.pup.edu.ph
12	Jun 05, 2026 05:23 AM	admin	VERIFY	Requirements	Administrator approved submitted requirements for student 2026-0001.

Figure 12. Activity logs page



Open University System

#	Username	Full Name / Email	Role	Status	Last Login	Actions
1	admin (you)	System Administrator admin@adams.local		Active	Never	Edit
2	staff	Admission Staff staff@adams.local		Active	Never	Edit, Deactivate

Figure 13. User management page

3.3 Functional Requirements and User Manual

The functional requirements below are written as user-manual actions so that the documentation is consistent with what the developed tool is expected to demonstrate during final evaluation.

Table 11. Functional requirements and user manual

Functional Requirement	User Manual Action	Expected System Response
FR-01 Login	User opens the ADAMS login page, enters username and password, and clicks Login.	System validates credentials, checks account status, opens the correct



		dashboard, and records last login when applicable.
FR-02 Logout	User clicks Logout from the sidebar or header.	System ends the session and returns the user to the login page.
FR-03 Password Reset	User clicks Forgot Password, enters the account email/username, and follows the reset process.	System generates a password reset token, stores expiration and usage status, and allows password update within the valid token period.
FR-04 Dashboard Monitoring	Authorized user opens Dashboard after login.	System displays student counts, requirement indicators, category information, and management summaries.
FR-05 Add Student	Superadmin or Staff opens Student Management, clicks Add Student, encodes profile information, selects category, and saves.	System validates required fields, stores the student record, and links the selected category.
FR-06 Search/View Student	User searches by student name, email, or category and opens the record.	System retrieves matching records and displays profile details and requirement status.
FR-07 Update Student	Authorized user opens a student record, edits allowed fields, and saves changes.	System updates the student table and records the action in activity_log.
FR-08 Archive Student	Authorized user clicks Archive for a student record and confirms the action.	System copies key student information to archived_students and preserves archive details.
FR-09 Manage Categories	Superadmin opens Categories, adds/edits/deletes category records, and manages sort order.	System updates student_categories and prevents duplicate category names.
FR-10 Manage Requirement Types	Superadmin opens Requirements, adds/edits/deletes requirement types and descriptions.	System updates requirement_types and prevents duplicate requirement names.
FR-11 Map Category Requirements	Superadmin assigns requirement types to the relevant student category.	System stores mappings in category_requirements so each category has the correct checklist.
FR-12 Upload Requirement File	Authorized user opens a student requirement, selects the file, and submits/upload updates.	System stores file metadata/content or path, uploader ID, status, upload date, and upload timestamp.
FR-13 Update Requirement Status	Authorized user changes status to PENDING, SUBMITTED, VERIFIED, APPROVED, INSUFFICIENT, or REJECTED.	System updates student_requirements and supports completeness monitoring.
FR-14 Print Student Report	User opens student report/print page and prints the selected record.	System generates a report, assigns a reference number if used, and stores print_logs.
FR-15 View Activity Logs	Superadmin opens Activity Logs and filters or reviews actions.	System displays action type, module, username, description, and timestamp.
FR-16 Manage Users	Superadmin opens User Management and manages Superadmin or Staff user accounts.	System creates, updates, activates, or deactivates user accounts according to the allowed roles: superadmin and staff.

3.4 Non-Functional Requirements

Table 12. Non-functional requirements



Requirement Area	Description
Accuracy	The database must prevent duplicate or inconsistent records through primary keys, foreign keys, unique constraints, and valid status values.
Usability	Screens must be accessible through a browser and organized by common registrar tasks.
Security	Only authorized users should access restricted functions; passwords should be stored as hashes and reset tokens should expire.
Auditability	Major user actions, uploads, archiving, and print activities should be traceable through logs.
Maintainability	Database tables and application modules should be named clearly so future developers can extend the system.
Backup and Recovery	Database dumps and backup tables may be used for restoration, but backup tables are separated from the normalized operational model.



Chapter 4. SQL Implementation

4.1 SQL Query Requirements

The following SQL queries satisfy the required three simple, four moderate, and three difficult SQL codes. Each query is aligned with an actual ADAMS database function and uses the normalized table names documented in the data dictionary.

4.2 Simple SQL Queries

Simple SQL 1: List Student Categories - Category Management

```
SELECT category_id, category_name, sort_order
FROM student_categories
ORDER BY sort_order, category_name;
```

category_id	category_name	sort_order
1	Freshman	1
2	Transferee	2
3	Returning	3

Explanation and relevance: This query retrieves the master list of student categories used to classify students.

Simple SQL 2: List Requirement Types - Requirement Setup

```
SELECT requirement_id, requirement_name, description
FROM requirement_types
ORDER BY requirement_name;
```

requirement_id	requirement_name	description
1	Birth Certificate	Proof of birth
2	Valid ID	Identity document
3	Transcript of Records	Academic record

Explanation and relevance: This query shows the standard list of admission requirement types.

Simple SQL 3: Show Active Users - User Management

```
SELECT user_id, username, full_name, role
FROM users
WHERE is_active = 1
ORDER BY role, username;
```



user_id	username	full_name	role
1	superadmin	System Superadmin	superadmin
5	staff1	Registrar Staff	staff

Explanation and relevance: This query identifies active users who can access the system.

4.3 Moderate SQL Queries

Moderate SQL 1: Student Profile with Category - Student Search

```
SELECT s.student_id,  
       s.student_name,  
       s.email,  
       s.phone,  
       c.category_name  
FROM students s  
LEFT JOIN student_categories c ON c.category_id = s.category_id  
WHERE s.student_id = 1;
```

student_id	student_name	email	phone	category_name
1	Juana Santita	sample@email.edu	09171234567	Freshman

Explanation and relevance: This query joins students with student_categories to show the classification of a student.

Moderate SQL 2: Requirements Required per Category - Requirement Checklist

```
SELECT c.category_name,  
       r.requirement_name,  
       r.description  
FROM category_requirements cr  
JOIN student_categories c ON c.category_id = cr.category_id  
JOIN requirement_types r ON r.requirement_id = cr.requirement_id  
WHERE c.category_id = 1  
ORDER BY r.requirement_name;
```

category_name	requirement_name	description
Freshman	Birth Certificate	Proof of birth
Freshman	Valid ID	Identity document

Explanation and relevance: This query shows which requirements apply to a selected student category.

Moderate SQL 3: Student Requirement Status - Requirement Upload Monitoring

```
SELECT s.student_id,  
       s.student_name,  
       r.requirement_name,
```



```
COALESCE(sr.status, 'PENDING') AS current_status,  
sr.file_name,  
sr.uploaded_at  
FROM students s  
JOIN student_requirements sr ON sr.student_id = CAST(s.student_id AS CHAR)  
JOIN requirement_types r ON r.requirement_id = sr.requirement_id  
WHERE s.student_id = 1  
ORDER BY r.requirement_name;
```

student_id	student_name	requirement_name	current_status
1	Juana Santita	Birth Certificate	SUBMITTED
1	Juana Santita	Valid ID	PENDING

Explanation and relevance: This query joins students, student_requirements, and requirement_types to display one student's requirement checklist and status.

Moderate SQL 4: Print Logs with Student and User - Report / Print Monitoring

```
SELECT pl.ref_no,  
s.student_name,  
pl.document_type,  
u.full_name AS printed_by,  
pl.printed_at  
FROM print_logs pl  
LEFT JOIN students s ON s.student_id = pl.student_id  
LEFT JOIN users u ON u.user_id = pl.printed_by  
ORDER BY pl.printed_at DESC;
```

ref_no	student_name	document_type	printed_by
MN0-001	Juana Santita	STUDENT_REPORT	Registrar Staff

Explanation and relevance: This query connects print records to students and users for accountability.

4.4 Difficult SQL Queries

Difficult SQL 1: Missing or Pending Requirements - Completeness Report

```
SELECT s.student_id,  
s.student_name,  
c.category_name,  
r.requirement_name,  
COALESCE(sr.status, 'MISSING') AS status_result  
FROM students s  
JOIN student_categories c ON c.category_id = s.category_id  
JOIN category_requirements cr ON cr.category_id = c.category_id  
JOIN requirement_types r ON r.requirement_id = cr.requirement_id  
LEFT JOIN student_requirements sr
```



```
ON sr.student_id = CAST(s.student_id AS CHAR)
AND sr.requirement_id = r.requirement_id
WHERE sr.student_requirement_id IS NULL
OR sr.status IS NULL
OR sr.status IN ('PENDING', 'INSUFFICIENT', 'REJECTED')
ORDER BY s.student_name, r.requirement_name;
```

student_id	student_name	requirement_name	status_result
1	Juana Santita	Valid ID	PENDING
2	Mark Santos	Birth Certificate	MISSING

Explanation and relevance: This difficult query compares required documents per category against actual student requirement records to identify missing or unresolved requirements.

Difficult SQL 2: Requirement Completion Summary per Student - Dashboard / Management Report

```
SELECT s.student_id,
       s.student_name,
       c.category_name,
       COUNT(cr.requirement_id) AS required_count,
       COUNT(sr.student_requirement_id) AS recorded_count,
       SUM(CASE WHEN sr.file_name IS NOT NULL THEN 1 ELSE 0 END) AS uploaded_count,
       SUM(CASE WHEN sr.status IN ('SUBMITTED','VERIFIED','APPROVED') THEN 1 ELSE 0 END) AS completed_count,
       COUNT(cr.requirement_id) - COUNT(sr.student_requirement_id) AS missing_count
FROM students s
JOIN student_categories c ON c.category_id = s.category_id
JOIN category_requirements cr ON cr.category_id = c.category_id
LEFT JOIN student_requirements sr
  ON sr.student_id = CAST(s.student_id AS CHAR)
  AND sr.requirement_id = cr.requirement_id
GROUP BY s.student_id, s.student_name, c.category_name
ORDER BY missing_count DESC, s.student_name;
```

student_id	student_name	required_count	uploaded_count	missing_count
1	Juana Santita	6	5	1
2	Mark Santos	6	3	3

Explanation and relevance: This query summarizes required, recorded, uploaded, completed, and missing requirements per student.

Difficult SQL 3: Pending Requirement Aging Report - Follow-up / Monitoring Report

```
WITH requirement_status AS (
  SELECT s.student_id,
         s.student_name,
```



```
r.requirement_name,  
sr.status,  
sr.uploaded_at,  
ROW_NUMBER() OVER (  
    PARTITION BY s.student_id, r.requirement_id  
    ORDER BY COALESCE(sr.uploaded_at, '1900-01-01') DESC  
) AS rn  
FROM students s  
JOIN category_requirements cr ON cr.category_id = s.category_id  
JOIN requirement_types r ON r.requirement_id = cr.requirement_id  
LEFT JOIN student_requirements sr  
    ON sr.student_id = CAST(s.student_id AS CHAR)  
    AND sr.requirement_id = r.requirement_id  
)  
SELECT student_id,  
    student_name,  
    requirement_name,  
    COALESCE(status, 'MISSING') AS status_result,  
    uploaded_at,  
    CASE  
        WHEN uploaded_at IS NULL THEN NULL  
        ELSE DATEDIFF(CURDATE(), DATE(uploaded_at))  
    END AS days_since_upload  
FROM requirement_status  
WHERE rn = 1  
    AND (status IS NULL OR status = 'PENDING')  
ORDER BY student_name, requirement_name;
```

student_id	student_name	requirement_name	status_result	days_since_upload
1	Juana Santita	Valid ID	PENDING	5
2	Mark Santos	Transcript	MISSING	NULL

Explanation and relevance: This query uses a CTE and window function to identify the latest requirement status and highlight pending or missing requirements for follow-up.

4.5 SQL Code / DDL Schema

SQL DDL Schema for ADAMS

```
CREATE DATABASE IF NOT EXISTS doc_admission_db;  
USE doc_admission_db;  
  
CREATE TABLE users (  
    user_id INT PRIMARY KEY AUTO_INCREMENT,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    password VARCHAR(100) NOT NULL,  
    email VARCHAR(100),  
    full_name VARCHAR(100),  
    phone VARCHAR(20),  
    is_active TINYINT(1) NOT NULL DEFAULT 1,  
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    last_login DATETIME,
```



```
role VARCHAR(20) NOT NULL,
CHECK (role IN ('superadmin','staff'))
);

CREATE TABLE student_categories (
category_id INT PRIMARY KEY AUTO_INCREMENT,
category_name VARCHAR(100) NOT NULL UNIQUE,
sort_order INT DEFAULT 0
);

CREATE TABLE students (
student_id INT PRIMARY KEY AUTO_INCREMENT,
user_id INT,
student_name VARCHAR(100),
last_name VARCHAR(100) NOT NULL,
first_name VARCHAR(100) NOT NULL,
middle_name VARCHAR(100),
birth_date DATE,
email VARCHAR(100),
category_id INT,
remarks VARCHAR(255),
phone VARCHAR(20),
UNIQUE KEY uk_students_name_bdate (student_name, birth_date),
UNIQUE KEY uk_student_fullname (last_name, first_name, middle_name, birth_date),
FOREIGN KEY (category_id) REFERENCES student_categories(category_id)
ON DELETE SET NULL ON UPDATE CASCADE,
FOREIGN KEY (user_id) REFERENCES users(user_id)
ON DELETE SET NULL ON UPDATE CASCADE
);

CREATE TABLE requirement_types (
requirement_id INT PRIMARY KEY AUTO_INCREMENT,
requirement_name VARCHAR(100) NOT NULL UNIQUE,
description VARCHAR(255)
);

CREATE TABLE category_requirements (
category_id INT NOT NULL,
requirement_id INT NOT NULL,
PRIMARY KEY (category_id, requirement_id),
FOREIGN KEY (category_id) REFERENCES student_categories(category_id)
ON DELETE CASCADE ON UPDATE CASCADE,
FOREIGN KEY (requirement_id) REFERENCES requirement_types(requirement_id)
ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE student_requirements (
student_requirement_id INT PRIMARY KEY AUTO_INCREMENT,
student_id VARCHAR(20) NOT NULL,
requirement_id INT NOT NULL,
file_name VARCHAR(255),
file_path VARCHAR(500),
file_content LONGBLOB,
mime_type VARCHAR(100),
file_size BIGINT,
uploaded_by INT,
status VARCHAR(50),
upload_date DATE,
uploaded_at DATETIME,
UNIQUE KEY uk_student_req (student_id, requirement_id),
INDEX idx_sr_requirement (requirement_id),
INDEX idx_sr_status (status),
INDEX idx_sr_composite (student_id, requirement_id, file_name)
```



```
);

CREATE TABLE documents (
    document_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id VARCHAR(20),
    document_type VARCHAR(50),
    file_path VARCHAR(255),
    tracking_number VARCHAR(50),
    status VARCHAR(20),
    date_submitted DATETIME,
    remarks VARCHAR(255),
    student_pk INT NOT NULL,
    FOREIGN KEY (student_pk) REFERENCES students(student_id)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE activity_log (
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    username VARCHAR(100) NOT NULL,
    action VARCHAR(50) NOT NULL,
    module VARCHAR(50) NOT NULL,
    description TEXT NOT NULL,
    log_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
    ON DELETE CASCADE,
    CHECK (action IN ('ADD', 'UPDATE', 'DELETE', 'REPLACE', 'UPLOAD', 'ARCHIVE')),
    CHECK (module IN ('Students', 'Users', 'Categories', 'Requirements', 'Uploads', 'Profile'))
);

CREATE TABLE print_logs (
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    ref_no VARCHAR(100) NOT NULL UNIQUE,
    student_id INT,
    document_type VARCHAR(50) NOT NULL,
    printed_by INT,
    printed_by_name VARCHAR(100),
    printed_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    campus_code VARCHAR(10) NOT NULL DEFAULT 'MN0',
    FOREIGN KEY (student_id) REFERENCES students(student_id)
    ON DELETE SET NULL,
    FOREIGN KEY (printed_by) REFERENCES users(user_id)
    ON DELETE SET NULL,
    CHECK (document_type IN ('STUDENT_REPORT', 'ALL_STUDENTS_REPORT'))
);

CREATE TABLE password_reset_tokens (
    id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    token VARCHAR(64) NOT NULL UNIQUE,
    expires_at DATETIME NOT NULL,
    used TINYINT(1) DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
    ON DELETE CASCADE
);

CREATE TABLE archived_students (
    archive_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT NOT NULL,
    student_name VARCHAR(255),
    last_name VARCHAR(100),
    first_name VARCHAR(100),
```



```
middle_name VARCHAR(100),
email VARCHAR(255),
phone VARCHAR(50),
birth_date DATE,
category_id INT,
remarks TEXT,
archived_at DATETIME DEFAULT CURRENT_TIMESTAMP,
archived_by INT
);
```



Chapter 5. Program Requirements and Deployment

5.1 Hardware Requirements

Table 13. Hardware requirements

Hardware Component	Recommended Requirement
Development / Demonstration Computer	Intel Core i3 / Ryzen 3 or higher; 4 GB RAM minimum, 8 GB recommended; at least 5 GB free storage; Windows 10/11, Linux, or macOS.
Application/Database Server for LAN Demo	Dual-core CPU or higher; 8 GB RAM recommended; 20 GB storage or higher; reliable LAN connection.
Client Device	Desktop/laptop with updated web browser and network access to the application server.
Backup Storage	USB drive, external drive, or cloud storage for source code, SQL dumps, screenshots, and project documentation.

5.2 Software Requirements

Table 14. Software requirements

Software Component	Recommended Requirement
Operating System	Windows 10/11, Linux, or macOS for development and testing.
Application Server	Apache Tomcat or compatible Java web server for JSP/Servlet deployment.
Programming Stack	Java, JSP/Servlets, Maven project structure, DAO/model/servlet classes.
Database Server	MySQL 8.0 or MariaDB compatible server.
Database Connector	MySQL Connector/J JDBC driver.
Development Tools	Java JDK, Apache Maven, IDE such as IntelliJ IDEA/Eclipse/NetBeans, MySQL Workbench or phpMyAdmin, and Git if available.
Browser	Google Chrome, Microsoft Edge, Mozilla Firefox, or equivalent modern browser.
ER Modeling / Diagram Tools	Draw.io, Lucidchart, MySQL Workbench, or equivalent ERD/EERD modeling tool.
Security Tools/Features	Password hashing utility, role-based accounts, reset token expiry, and activity logging.

5.3 Deployment and Backup Requirements

- Create the database `doc_admission_db` and run the ADAMS DDL/schema script before launching the application.
- Configure the database connection settings in the application according to the local MySQL/MariaDB username, password, host, and port.
- Deploy the compiled web application to Apache Tomcat or run it through the configured development environment.



- Test login, dashboard, student management, categories, requirements, uploads, reports, activity logs, and user management before final presentation.
- Keep a copy of the source code package, database dump, executable/deployable files, and final documentation in both USB and cloud storage as suggested by the class requirement.

References

Polytechnic University of the Philippines Open University System. (2024). CAEPUP-OUS application form. Internal admission document.

Polytechnic University of the Philippines. (2026). DBMS project final documentation format and project requirements. Course requirement materials.

ADAMS project team. (2026). Admission Document and Applicant Management System source code and database package. Unpublished final project files.



Appendices

Appendix A. Source Code and Database Package Contents

The submitted compressed package includes the ADAMS application source code and database files. The following summary helps the panel verify that the documentation is consistent with the actual project package.

Table 15. Source code and database package contents

Package Area	File/Folder Reference	Purpose
Main application folder	ADAMS - VANDAM/doc_ad_sys	Contains Java web application files, Maven structure, JSP pages, servlets, DAO classes, models, utilities, and web assets.
Database folder	ADAMS_db	Contains SQL dump files for the operational database tables.
Java DAO classes	dao/*.java	Contains data access classes for users, students, requirements, uploads, logs, dashboard, printing, and archives.
Java model classes	model/*.java	Contains object representations of users, students, categories, requirements, logs, and uploaded files.
Servlet classes	servlet/*.java	Handles login, dashboard, student management, requirements, uploads, reports, verification, and user management workflows.
JSP pages	login.jsp, dashboard.jsp, students.jsp, uploads.jsp, archives.jsp, categories.jsp, requirements.jsp, activity-log.jsp, user-management.jsp	Presents the web user interface pages shown in the screenshots.
Database dump files	doc_admission_db_users.sql, students.sql, student_categories.sql, requirement_types.sql, category_requirements.sql, student_requirements.sql, documents.sql, activity_log.sql, print_logs.sql, password_reset_tokens.sql, archived_students.sql	Provides the SQL table data/structure used for the ADAMS database package.
Backup/support file	students_backup	Excluded from the normalized operational model because it is a support/backup table.